

XL Logic Beginner Programming Guide

This guide is the first beginner-friendly entry point for programming XL Logic computers. It keeps the examples in plain Python, but it starts with the simplest helpers instead of the full low-level API.

The mod now also ships an in-game guide book item. Right-click it to open the same beginner-oriented concepts, code examples, and troubleshooting hints directly in Minecraft.

You can also open the guide directly from the computer screen through the Guide button in the header. The same header also exposes a Builder button which opens a no-code block editor for players who do not want to write Python by hand.

The Builder now also ships with ready-made templates such as discovery, live clock, rain watch, item checks, fluid tank viewers, source-to-sink item routes, source-to-sink fluid routes, rain-controlled lamps, stock alerts, night lamps, thunder warnings, strong-rain alerts, and evening lights.

It now also includes simple logic guards, Otherwise branches, and basic comparisons so one block can run only when world day or night, thunder, rain strength, moon phase, custom time windows, preset windows like dawn or evening, redstone, item counts, or fluid amounts match a chosen condition.

1. Mental Model

An XL Logic computer runs Python on the server.

- Your code can read devices on the local cable segment.
- Some devices can also appear through XLAPI bridges as read-only remote devices.
- Output is shown on the computer and on linked screen blocks, and named screens can be addressed individually once you give them endpoint names.
- Long-running programs must yield back to the server between ticks.

The most important beginner rule is this:

- Use `yield from pause(1)` inside manual loops.
- Use `yield from repeat(step, 1)` when you want the same step to run every tick.

If you forget to yield in a long-running loop, the runtime watchdog will stop the program.

Safety Boundary

An XL Logic script is not a general-purpose host Python script.

- It can use the XL Logic runtime objects such as world, screen, network, output, and devices.
- It cannot access the host OS, local files, spawned processes, raw sockets, Java imports, or polyglot host APIs.
- It also does not expose a direct path to admin commands such as `op`, `restart`, or `shutdown`.

In practice, this means you automate the Minecraft world through the provided APIs instead of reaching outside the server process.

2. Beginner Helpers

The beginner layer adds two simple objects and a few convenience functions.

screen object

Use screen for visible output.

- `screen.print(text)`
- `screen.show(title, fields)`
- `screen.table(title, columns, rows)`
- `screen.plan(title, fields)`

Example:

```
screen.print("Hello from XL Logic")

screen.show("World", {
    "day_time": world.day_time(),
    "raining": world.is_raining(),
})
```

Named screen blocks expose the same visible helpers after you name them and fetch them through `get_device(name)`.

```
left_panel = get_device("left_panel")
if left_panel is not None and world.available():
    left_panel.show("World", {
        "day_time": world.day_time(),
        "raining": world.is_raining(),
    })
```

network object

Use network to discover devices without remembering the lower-level registry calls.

- `network.names()`
- `network.names("clock")`
- `network.all()`
- `network.all("material_io")`
- `network.get("timekeeper")`
- `network.require("timekeeper")`
- `network.find("clock")`
- `network.types()`

Example:

```
screen.show("Device types", {
    "types": ", ".join(network.types()),
    "count": len(network.names()),
})
```

Convenience functions

These helpers call the objects above for you.

- say(text)
- show(title, fields)
- device(name)
- require_device(name)
- find_device(device_type)
- list_device_names(device_type=None)
- devices_by_type(device_type)
- pause(ticks=1)
- repeat(step, every_ticks=1)

3. Important Global Data

These globals are always available.

- computer: a simple dictionary with name, position, and endpoint_count
- endpoints: metadata for all visible endpoints
- world: live world API
- screen: beginner output object
- network: beginner device lookup object

The advanced objects are still available too:

- output
- devices
- get_device(...)
- show_kv(...)
- show_table(...)
- show_plan_card(...)

Use the advanced layer when you need the full API surface.

4. First Programs

Hello World

```
screen.print("Hello, world")
```

Show Computer and World Information

```
screen.show("Computer", {
    "name": computer["name"],
    "position": computer["position"],
    "endpoints": computer["endpoint_count"],
})

if world.available():
    screen.show("World", {
        "dimension": world.dimension(),
        "day_time": world.day_time(),
        "raining": world.is_raining(),
    })
```

List All Visible Devices

```
rows = []
for name in list_device_names():
    current = device(name)
    rows.append([name, current.type(), current.network_scope(),
current.remote_policy()])

screen.table("Devices", ["API", "Type", "Scope", "Policy"], rows)
```

5. Long-Running Programs

Manual Loop with pause

Use this when you want full control over the loop.

```
while True:
    if world.available():
        screen.show("World", {
            "day_time": world.day_time(),
            "raining": world.is_raining(),
        })
    yield from pause(1)
```

Repeating Step with repeat

Use this when the same function should run every tick.

```

last_rain = None

def tick():
    global last_rain
    if not world.available():
        return
    raining = world.is_raining()
    if raining != last_rain:
        screen.print("Raining: " + str(raining))
        last_rain = raining

yield from repeat(tick, 1)

```

6. Working With Common Device Types

You can ask the network for the first visible device of a type.

Clock

```

clock = find_device("clock")
if clock is not None:
    screen.show("Clock", {
        "game_time": clock.game_time(),
        "day_time": clock.day_time(),
        "real_time": clock.real_time(),
    })

```

Rain Sensor

```

sensor = find_device("rain_sensor")
if sensor is not None:
    screen.show("Rain Sensor", {
        "raining": sensor.is_raining(),
        "rain_level": sensor.rain_level(),
    })

```

Redstone I/O

```

redstone = find_device("redstone_io")
if redstone is not None:
    level = redstone.read("north")
    screen.show("North Signal", {"level": level})

```

If you configured side aliases, you can use them instead of the canonical side names.

Material I/O

```
storage = find_device("material_io")
if storage is not None:
    screen.table(
        "North Inventory",
        ["Item", "Count"],
        [[stack["item"], stack["count"]] for stack in
storage.inventory("north")],
    )
```

If you named two Material I/O endpoints, you can also route directly between them.

```
source = get_device("source_io")
if source is not None:
    moved_items = source.transfer_item_to("sink_io", "south", "south", 0, 16)
    moved_fluid = source.transfer_fluid_to("sink_io", "south", "south", 0,
1000)
    screen.show("Route", {
        "items": moved_items,
        "fluid": moved_fluid,
    })
```

7. Device Types You Will See Often

These are the most common beginner-facing types.

- clock
- light_sensor
- rain_sensor
- redstone_io
- material_io
- crafting_io
- crafting_cpu
- xlapl_block

Call `network.types()` if you want to inspect the types that are currently visible in your setup.

8. Recommended Learning Path

Start in this order:

1. `screen.print` and `screen.show`
2. `list_device_names` and `device(...)`
3. `find_device("clock")` or `find_device("rain_sensor")`

4. pause and repeat
5. redstone_io and material_io
6. crafting_io and crafting_cpu

This keeps the first programs short and avoids the full API surface until you actually need it.

9. When To Drop Down To The Advanced API

Use the advanced layer if you need one of these:

- exact API names with `get_device(...)`
- direct access to the device registry via `devices`
- rich output helpers like `show_table` and `show_plan_card`
- detailed per-device methods that are not covered by the beginner examples

The beginner layer is only a friendly wrapper. It does not replace the advanced API.

10. Troubleshooting

If a program does not behave as expected, check these points first.

- The device may not be visible on the same local cable segment.
- The device may exist but not be available right now.
- A bridged remote device may be read-only.
- A long-running loop may be missing `yield` from `pause(...)` or `yield` from `repeat(...)`.
- The endpoint name in your script may not match the actual API name.

This quick probe is useful when debugging:

```
screen.show("Debug", {
    "device_count": len(list_device_names()),
    "device_types": ", ".join(network.types()),
    "world_available": world.available(),
})
```